

# Hospital Appointment and Patient Care Management System (HAPCMS)

DBMS Course Project Report

**Student Name:** Shaik Imaduddin

**Roll Number:** 25WU0101048

**Course:** Database Management Systems (DBMS)

**Academic Year:** 2025-2029

**Project Type:** DBMS Course Project

**University:** Woxsen University

**Branch:** CSE - AIML

## 2. Abstract

The Hospital Appointment and Patient Care Management System (HAPCMS) is a database-driven web application developed to manage departments, physicians, schedules, patients, appointments, clinical consultations, electronic medical records (EMR), diagnostic laboratory tests, inpatient ward beds, admissions, billing, and payments in a structured relational system. The application uses PostgreSQL 16 (hosted on Neon Cloud) as the database layer, Node.js with Next.js 14 (App Router) and Prisma ORM for backend data access and transactions, and React with Tailwind CSS and Lucide Icons for the user interface. The system supports patient registration, conflict-free doctor appointment scheduling, consultation logging, diagnostic test ordering and tracking, real-time inpatient bed occupancy monitoring, itemized billing, and multi-tender payment reconciliation. Transaction-based writes keep related database records consistent, while primary keys, foreign keys, domain CHECK rules, custom ENUM types, and composite unique indexes protect data integrity.

## 3. Introduction and Problem Statement

### 3.1 Introduction

A healthcare institution produces interrelated data continuously: patients register and schedule outpatient visits, doctors examine patients and record clinical symptoms, diagnoses, and multi-drug prescriptions, laboratory technicians perform ordered diagnostics, nursing staff manage inpatient bed allocations and admissions, and cashiers generate itemized bills settled by various tender methods. Managing these records across disparate spreadsheets or isolated legacy software creates duplication, scheduling overlaps, unrecorded bed allocations, and billing leakage. This project models the complete healthcare lifecycle using a 3NF-normalized relational database and a connected web interface.

### 3.2 Problem Statement

The hospital requires a reliable system to maintain doctor rosters, accept and track outpatient appointments, record EMR consultations, manage inpatient ward capacities, track laboratory tests, generate itemized bills, record payments, and produce management reports. The system must preserve relationships between records and prevent invalid operations, such as double-booking a physician for the same time slot, assigning an already occupied bed, admitting patients with chronologically impossible discharge dates, entering non-positive tariffs or quantities, or recording payments that exceed the invoiced balance.

## 4. Objectives and Scope

### 4.1 Objectives

- Maintain departments, doctors, recurring weekly schedules, patients, appointments, consultations, diagnoses, prescriptions, prescription items, lab tests, test orders, wards, beds, admissions, bills, and payments in PostgreSQL 16.
- Provide a fully normalized Third Normal Form (3NF) relational architecture across 16 tables.
- Guarantee zero appointment double-booking collisions through composite unique indexing on (doctor\_id, appointment\_date, appointment\_time).
- Manage inpatient bed allocations with strictly controlled status transitions (AVAILABLE, OCCUPIED, MAINTENANCE).

- Enforce domain integrity via custom PostgreSQL ENUM types and CHECK constraints for fees, quantities, and dates.
- Execute multi-table writes atomically across appointments, admissions, and payments to guarantee ACID compliance.
- Generate comprehensive management reports using multi-table joins, aggregations, window functions, and views.
- Deliver a responsive, accessible clinical management web interface using Next.js 14 and Prisma ORM.

## 4.2 Scope

- Provider and schedule roster management with capacity caps.
- Patient directory with longitudinal medical histories.
- Calendar-based appointment scheduling with live collision prevention.
- Clinical consultation, ICD-10 diagnostic coding, and electronic prescriptions.
- Diagnostic laboratory test catalog, specimen collection, and abnormal result flagging.
- Inpatient ward bed management and admission-to-discharge workflow.
- Consolidated itemized billing and multi-tender payment ledger.
- Executive reporting for longitudinal summaries, bed occupancy, doctor utilization, and departmental revenue.
- Local and cloud-connected deployment using PostgreSQL 16, Prisma ORM, and Next.js 14.

## 5. Software and Hardware Requirements

The development and runtime requirements for the system are summarized in the following table:

| Category             | Requirement / Specification                                    |
|----------------------|----------------------------------------------------------------|
| Operating System     | Windows 10/11 / macOS / Linux                                  |
| Backend Framework    | Next.js 14.2 (App Router, Node.js Server Actions)              |
| Database Engine      | PostgreSQL 16 (Neon Cloud Serverless, SSL-encrypted)           |
| ORM / DB Client      | Prisma ORM 6.4.1 / pg (Node.js PostgreSQL client)              |
| Frontend & Styling   | HTML5, React 18, Tailwind CSS, Lucide Icons                    |
| Runtime Environments | Node.js v20+, TypeScript 5, Python 3.11+                       |
| Database Tooling     | Prisma Studio (Visual GUI), psql interactive CLI               |
| Testing Framework    | Prisma Client test scripts, Jest / Vitest                      |
| Browser Support      | Modern Chromium (Chrome, Edge) / Firefox / Safari              |
| Recommended Hardware | Dual-core or Quad-core processor, 8 GB RAM, 10 GB free storage |

## 6. ER Diagram

The ER diagram represents the sixteen entities used by the Hospital Appointment and Patient Care Management System (HAPCMS) and their relationships across six functional clusters: Provider & Roster, Patient & Appointment, Clinical EMR, Diagnostic Laboratory, Inpatient Care, and Financial Ledger.

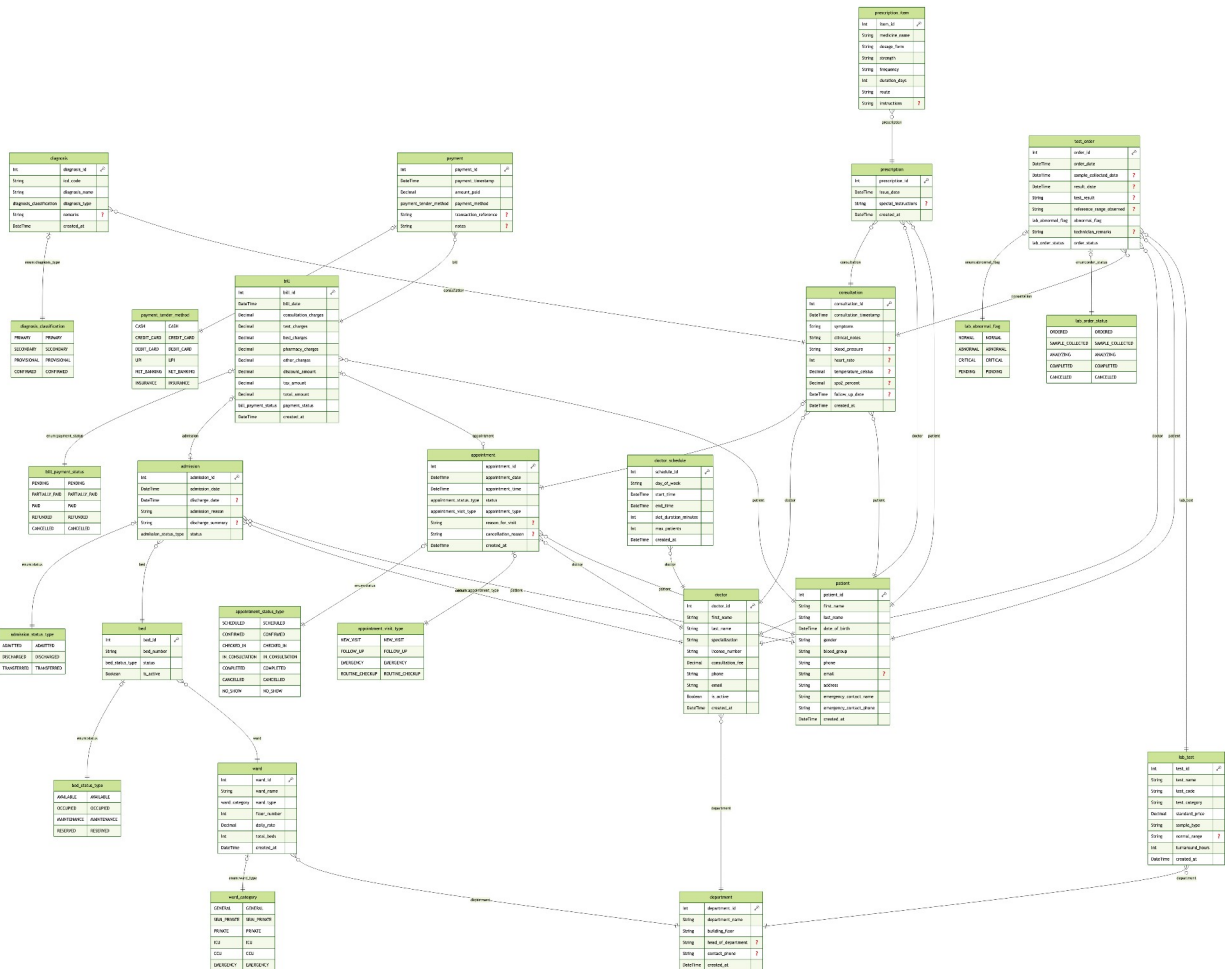


Figure: Entity-Relationship Diagram for HAPCMS (16 Entities)

## 7. Relational Schema and Normalization

### 7.1 Relational Schema

The tables below reproduce the complete structural schema of the implemented database, detailing Field name, Data Type, Nullability, Key classification, Default value, and Relational Constraints/Descriptions.

#### 7.1.1 Department

| Field              | Type         | Null | Key | Default           | Extra / Notes             |
|--------------------|--------------|------|-----|-------------------|---------------------------|
| department_id      | serial / int | NO   | PRI | auto_increment    | Primary Key               |
| department_name    | varchar(100) | NO   | UNI | NULL              | Unique Department Title   |
| building_floor     | varchar(50)  | NO   |     | NULL              | Building & Floor Location |
| head_of_department | varchar(100) | YES  |     | NULL              | Physician HOD / Lead      |
| contact_phone      | varchar(20)  | YES  |     | NULL              | Department Extension      |
| created_at         | timestamptz  | NO   |     | CURRENT_TIMESTAMP | Creation Timestamp        |

#### 7.1.2 Doctor

| Field            | Type          | Null | Key | Default           | Extra / Notes                        |
|------------------|---------------|------|-----|-------------------|--------------------------------------|
| doctor_id        | serial / int  | NO   | PRI | auto_increment    | Primary Key                          |
| department_id    | int           | NO   | FK  | NULL              | References department(department_id) |
| first_name       | varchar(50)   | NO   |     | NULL              | Doctor First Name                    |
| last_name        | varchar(50)   | NO   |     | NULL              | Doctor Last Name                     |
| specialization   | varchar(100)  | NO   |     | NULL              | Medical Specialty                    |
| license_number   | varchar(50)   | NO   | UNI | NULL              | Medical Council License              |
| phone            | varchar(20)   | NO   |     | NULL              | Direct Telephone                     |
| email            | varchar(100)  | NO   | UNI | NULL              | Institutional Email                  |
| consultation_fee | numeric(10,2) | NO   |     | NULL              | CHECK (consultation_fee >= 0)        |
| is_active        | boolean       | NO   |     | TRUE              | Active Hospital Roster State         |
| created_at       | timestamptz   | NO   |     | CURRENT_TIMESTAMP | Record Creation Timestamp            |

#### 7.1.3 DoctorSchedule

| Field       | Type         | Null | Key | Default        | Extra / Notes                 |
|-------------|--------------|------|-----|----------------|-------------------------------|
| schedule_id | serial / int | NO   | PRI | auto_increment | Primary Key                   |
| doctor_id   | int          | NO   | FK  | NULL           | References doctor(doctor_id)  |
| day_of_week | varchar(15)  | NO   |     | NULL           | Duty Day (MONDAY-SUNDAY)      |
| start_time  | time         | NO   |     | NULL           | Shift Start Time              |
| end_time    | time         | NO   |     | NULL           | CHECK (end_time > start_time) |

| Field        | Type | Null | Key | Default | Extra / Notes            |
|--------------|------|------|-----|---------|--------------------------|
| max_patients | int  | NO   |     | NULL    | CHECK (max_patients > 0) |

#### 7.1.4 Patient

| Field                   | Type         | Null | Key | Default           | Extra / Notes                 |
|-------------------------|--------------|------|-----|-------------------|-------------------------------|
| patient_id              | serial / int | NO   | PRI | auto_increment    | Primary Key                   |
| first_name              | varchar(50)  | NO   |     | NULL              | Patient Given Name            |
| last_name               | varchar(50)  | NO   |     | NULL              | Patient Family Name           |
| date_of_birth           | date         | NO   |     | NULL              | CHECK (dob <= CURRENT_DATE)   |
| gender                  | varchar(10)  | NO   |     | NULL              | ENUM: MALE, FEMALE, OTHER     |
| blood_group             | varchar(5)   | YES  |     | NULL              | CHECK (IN valid ABO/Rh types) |
| phone                   | varchar(20)  | NO   | UNI | NULL              | Primary Contact Number        |
| email                   | varchar(100) | YES  |     | NULL              | Email Address                 |
| address                 | text         | YES  |     | NULL              | Residential Address           |
| emergency_contact_name  | varchar(100) | YES  |     | NULL              | Emergency Next-of-Kin         |
| emergency_contact_phone | varchar(20)  | YES  |     | NULL              | Emergency Contact Phone       |
| created_at              | timestampz   | NO   |     | CURRENT_TIMESTAMP | Registration Timestamp        |

#### 7.1.5 Appointment

| Field               | Type         | Null | Key | Default           | Extra / Notes                          |
|---------------------|--------------|------|-----|-------------------|----------------------------------------|
| appointment_id      | serial / int | NO   | PRI | auto_increment    | Primary Key                            |
| patient_id          | int          | NO   | FK  | NULL              | References patient(patient_id) CASCADE |
| doctor_id           | int          | NO   | FK  | NULL              | References doctor(doctor_id) RESTRICT  |
| appointment_date    | date         | NO   |     | NULL              | Scheduled Consultation Date            |
| appointment_time    | time         | NO   |     | NULL              | Scheduled Consultation Time            |
| visit_type          | varchar(20)  | NO   |     | 'NEW_VISIT'       | ENUM: NEW_VISIT, FOLLOW_UP...          |
| status              | varchar(20)  | NO   |     | 'SCHEDULED'       | ENUM: SCHEDULED, COMPLETED...          |
| reason_for_visit    | text         | YES  |     | NULL              | Chief Medical Complaint                |
| cancellation_reason | text         | YES  |     | NULL              | Cancellation Justification             |
| created_at          | timestampz   | NO   |     | CURRENT_TIMESTAMP | Booking Timestamp                      |
| uq_doctor_slot      | composite    | NO   | UNI | UNIQUE            | UNIQUE (doctor_id, date, time)         |

### 7.1.6 Consultation

| Field             | Type         | Null | Key     | Default           | Extra / Notes                |
|-------------------|--------------|------|---------|-------------------|------------------------------|
| consultation_id   | serial / int | NO   | PRI     | auto_increment    | Primary Key                  |
| appointment_id    | int          | NO   | UNI, FK | NULL              | References appointment (1:1) |
| consultation_date | timestampz   | NO   |         | CURRENT_TIMESTAMP | Clinical Encounter Timestamp |
| symptoms          | text         | NO   |         | NULL              | Documented Patient Symptoms  |
| clinical_notes    | text         | YES  |         | NULL              | Physician Examination Notes  |
| follow_up_date    | date         | YES  |         | NULL              | Recommended Follow-up Date   |
| created_at        | timestampz   | NO   |         | CURRENT_TIMESTAMP | Record Timestamp             |

### 7.1.7 Diagnosis

| Field           | Type         | Null | Key | Default        | Extra / Notes                            |
|-----------------|--------------|------|-----|----------------|------------------------------------------|
| diagnosis_id    | serial / int | NO   | PRI | auto_increment | Primary Key                              |
| consultation_id | int          | NO   | FK  | NULL           | References consultation(consultation_id) |
| icd_code        | varchar(20)  | NO   |     | NULL           | ICD-10 Diagnostic Code                   |
| diagnosis_name  | varchar(255) | NO   |     | NULL           | Disease / Condition Name                 |
| diagnosis_type  | varchar(20)  | NO   |     | 'PRIMARY'      | ENUM: PRIMARY, SECONDARY...              |
| remarks         | text         | YES  |     | NULL           | Clinical Remarks                         |

### 7.1.8 Prescription

| Field             | Type         | Null | Key | Default           | Extra / Notes                            |
|-------------------|--------------|------|-----|-------------------|------------------------------------------|
| prescription_id   | serial / int | NO   | PRI | auto_increment    | Primary Key                              |
| consultation_id   | int          | NO   | FK  | NULL              | References consultation(consultation_id) |
| prescription_date | timestampz   | NO   |     | CURRENT_TIMESTAMP | Prescription Generation Date             |
| instructions      | text         | YES  |     | NULL              | Pharmaceutical Regimen Notes             |
| created_at        | timestampz   | NO   |     | CURRENT_TIMESTAMP | Creation Timestamp                       |

### 7.1.9 PrescriptionItem

| Field           | Type         | Null | Key | Default        | Extra / Notes                            |
|-----------------|--------------|------|-----|----------------|------------------------------------------|
| item_id         | serial / int | NO   | PRI | auto_increment | Primary Key                              |
| prescription_id | int          | NO   | FK  | NULL           | References prescription(prescription_id) |
| medicine_name   | varchar(100) | NO   |     | NULL           | Medication Name                          |
| dosage          | varchar(50)  | NO   |     | NULL           | E.g. '1 Tablet', '5 ml'                  |
| strength        | varchar(50)  | NO   |     | NULL           | E.g. '500 mg', '10 mg'                   |

| Field         | Type        | Null | Key | Default | Extra / Notes                 |
|---------------|-------------|------|-----|---------|-------------------------------|
| frequency     | varchar(50) | NO   |     | NULL    | E.g. 'Twice daily after food' |
| duration_days | int         | NO   |     | NULL    | CHECK (duration_days > 0)     |
| quantity      | int         | NO   |     | NULL    | CHECK (quantity > 0)          |
| remarks       | text        | YES  |     | NULL    | Administration Warnings       |

#### 7.1.10 LabTest

| Field            | Type          | Null | Key | Default        | Extra / Notes                |
|------------------|---------------|------|-----|----------------|------------------------------|
| test_id          | serial / int  | NO   | PRI | auto_increment | Primary Key                  |
| test_code        | varchar(20)   | NO   | UNI | NULL           | Procedural Lab Code          |
| test_name        | varchar(100)  | NO   |     | NULL           | Investigation Name           |
| category         | varchar(50)   | NO   |     | NULL           | Pathology, Biochemistry...   |
| sample_type      | varchar(50)   | NO   |     | NULL           | Blood, Serum, Urine, Imaging |
| standard_price   | numeric(10,2) | NO   |     | NULL           | CHECK (standard_price >= 0)  |
| normal_range     | varchar(100)  | YES  |     | NULL           | Clinical Benchmark Baseline  |
| turnaround_hours | int           | NO   |     | NULL           | CHECK (turnaround_hours > 0) |
| is_active        | boolean       | NO   |     | TRUE           | Active Catalog State         |

#### 7.1.11 TestOrder

| Field                 | Type         | Null | Key | Default           | Extra / Notes                     |
|-----------------------|--------------|------|-----|-------------------|-----------------------------------|
| order_id              | serial / int | NO   | PRI | auto_increment    | Primary Key                       |
| patient_id            | int          | NO   | FK  | NULL              | References patient(patient_id)    |
| consultation_id       | int          | YES  | FK  | NULL              | References consultation           |
| ordered_by_doctor_id  | int          | NO   | FK  | NULL              | References doctor(doctor_id)      |
| test_id               | int          | NO   | FK  | NULL              | References lab_test(test_id)      |
| order_date            | timestampz   | NO   |     | CURRENT_TIMESTAMP | Requisition Timestamp             |
| order_status          | varchar(25)  | NO   |     | 'ORDERED'         | ENUM: ORDERED, COMPLETED...       |
| sample_collected_at   | timestampz   | YES  |     | NULL              | Specimen Collection Time          |
| result_available_at   | timestampz   | YES  |     | NULL              | Authorization Timestamp           |
| test_result           | text         | YES  |     | NULL              | Quantitative / Qualitative Result |
| normal_range_snapshot | varchar(100) | YES  |     | NULL              | Reference Baseline at Test Time   |
| abnormal_flag         | varchar(15)  | NO   |     | 'NORMAL'          | ENUM: NORMAL, ABNORMAL, CRITICAL  |

| Field              | Type | Null | Key | Default | Extra / Notes            |
|--------------------|------|------|-----|---------|--------------------------|
| technician_remarks | text | YES  |     | NULL    | Pathologist Observations |

#### 7.1.12 Ward

| Field      | Type          | Null | Key | Default           | Extra / Notes               |
|------------|---------------|------|-----|-------------------|-----------------------------|
| ward_id    | serial / int  | NO   | PRI | auto_increment    | Primary Key                 |
| ward_name  | varchar(100)  | NO   | UNI | NULL              | Ward Designation (e.g. ICU) |
| ward_type  | varchar(50)   | NO   |     | NULL              | ENUM: GENERAL, ICU, CCU...  |
| total_beds | int           | NO   |     | NULL              | CHECK (total_beds > 0)      |
| daily_rate | numeric(10,2) | NO   |     | NULL              | CHECK (daily_rate >= 0)     |
| created_at | timestamptz   | NO   |     | CURRENT_TIMESTAMP | Creation Timestamp          |

#### 7.1.13 Bed

| Field       | Type         | Null | Key | Default        | Extra / Notes                |
|-------------|--------------|------|-----|----------------|------------------------------|
| bed_id      | serial / int | NO   | PRI | auto_increment | Primary Key                  |
| ward_id     | int          | NO   | FK  | NULL           | References ward(ward_id)     |
| bed_number  | varchar(20)  | NO   |     | NULL           | Bed Code (e.g. ICU-01)       |
| bed_type    | varchar(50)  | NO   |     | NULL           | ENUM: STANDARD, ICU...       |
| status      | varchar(20)  | NO   |     | 'AVAILABLE'    | ENUM: AVAILABLE, OCCUPIED... |
| uq_ward_bed | composite    | NO   | UNI | UNIQUE         | UNIQUE (ward_id, bed_number) |

#### 7.1.14 Admission

| Field               | Type         | Null | Key | Default           | Extra / Notes                  |
|---------------------|--------------|------|-----|-------------------|--------------------------------|
| admission_id        | serial / int | NO   | PRI | auto_increment    | Primary Key                    |
| patient_id          | int          | NO   | FK  | NULL              | References patient(patient_id) |
| admitting_doctor_id | int          | NO   | FK  | NULL              | References doctor(doctor_id)   |
| bed_id              | int          | NO   | FK  | NULL              | References bed(bed_id)         |
| admission_date      | timestamptz  | NO   |     | CURRENT_TIMESTAMP | Admission Timestamp            |
| discharge_date      | timestamptz  | YES  |     | NULL              | CHECK (discharge >= admission) |
| admission_reason    | text         | NO   |     | NULL              | Clinical Reason for Admission  |
| discharge_summary   | text         | YES  |     | NULL              | Physician Discharge Summary    |
| status              | varchar(20)  | NO   |     | 'ADMITTED'        | ENUM: ADMITTED, DISCHARGED...  |

### 7.1.15 Bill

| Field          | Type          | Null | Key | Default           | Extra / Notes                    |
|----------------|---------------|------|-----|-------------------|----------------------------------|
| bill_id        | serial / int  | NO   | PRI | auto_increment    | Primary Key                      |
| patient_id     | int           | NO   | FK  | NULL              | References patient(patient_id)   |
| appointment_id | int           | YES  | FK  | NULL              | References appointment           |
| admission_id   | int           | YES  | FK  | NULL              | References admission             |
| bill_date      | timestampz    | NO   |     | CURRENT_TIMESTAMP | Invoice Generation Date          |
| due_date       | date          | NO   |     | NULL              | Invoice Due Date                 |
| total_amount   | numeric(10,2) | NO   |     | NULL              | CHECK (total_amount >= 0)        |
| payment_status | varchar(20)   | NO   |     | 'PENDING'         | ENUM: PENDING, PAID, REFUNDED... |
| created_at     | timestampz    | NO   |     | CURRENT_TIMESTAMP | Timestamp                        |

### 7.1.16 Payment

| Field                 | Type          | Null | Key | Default           | Extra / Notes                    |
|-----------------------|---------------|------|-----|-------------------|----------------------------------|
| payment_id            | serial / int  | NO   | PRI | auto_increment    | Primary Key                      |
| bill_id               | int           | NO   | FK  | NULL              | References bill(bill_id)         |
| payment_date          | timestampz    | NO   |     | CURRENT_TIMESTAMP | Settlement Timestamp             |
| amount_paid           | numeric(10,2) | NO   |     | NULL              | CHECK (amount_paid > 0)          |
| payment_method        | varchar(30)   | NO   |     | NULL              | ENUM: CASH, UPI, CARD, INSURANCE |
| transaction_reference | varchar(100)  | YES  | UNI | NULL              | Banking / Gateway Ref            |
| notes                 | text          | YES  |     | NULL              | Ledger Remarks                   |
| created_at            | timestampz    | NO   |     | CURRENT_TIMESTAMP | Timestamp                        |

## 7.2 Normalization

The schema strictly satisfies First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) principles:

1. First Normal Form (1NF): All attributes contain atomic, indivisible values. Repeating groups and multivalued arrays have been eliminated by creating separate child relations (e.g., individual prescription line items are decomposed into distinct rows in prescription\_item; diagnostic findings are decomposed into individual diagnosis rows referencing standard ICD-10 codes).
2. Second Normal Form (2NF): The schema is in 1NF and contains no partial functional dependencies. Every relation uses a single-column surrogate primary key (SERIAL integer). Because no candidate key is composite, no non-prime attribute can depend on a proper subset of a candidate key; each non-prime attribute is fully functionally dependent on the entire primary key.
3. Third Normal Form (3NF): The schema is in 2NF and contains no transitive functional dependencies ( $X \rightarrow Y$  where Y is non-prime and X is not a superkey). Departmental floor locations reside solely in department rather

than doctor; ward daily rates reside exclusively in ward rather than bed; and consultation fee schedules reside in doctor rather than appointment or bill. Thus, all functional dependencies are strictly of the form X -> A where X is a candidate key, certifying 3NF compliance.

## 8. SQL Commands Used (DDL, DML) with Sample Outputs

### 8.1 Database and Table Creation (DDL Scripts)

```
-- 1. Custom Domain ENUM Types
CREATE TYPE appointment_status_type AS ENUM (
    'SCHEDULED', 'CONFIRMED', 'CHECKED_IN', 'IN_CONSULTATION', 'COMPLETED', 'CANCELLED',
    'NO_SHOW'
);
CREATE TYPE bed_status_type AS ENUM ('AVAILABLE', 'OCCUPIED', 'MAINTENANCE', 'RESERVED');
CREATE TYPE bill_payment_status AS ENUM ('PENDING', 'PARTIALLY_PAID', 'PAID', 'REFUNDED',
    'CANCELLED');

-- 2. Core Master Table: Department
CREATE TABLE department (
    department_id SERIAL PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE,
    building_floor VARCHAR(50) NOT NULL,
    head_of_department VARCHAR(100),
    contact_phone VARCHAR(20),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 3. Doctor Table with Domain CHECK and Unique Constraints
CREATE TABLE doctor (
    doctor_id SERIAL PRIMARY KEY,
    department_id INTEGER NOT NULL REFERENCES department(department_id) ON DELETE RESTRICT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    specialization VARCHAR(100) NOT NULL,
    license_number VARCHAR(50) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    consultation_fee NUMERIC(10,2) NOT NULL CHECK (consultation_fee >= 0.00),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 4. Appointment Table with Zero-Collision Unique Constraint
CREATE TABLE appointment (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL REFERENCES patient(patient_id) ON DELETE CASCADE,
    doctor_id INTEGER NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    visit_type VARCHAR(20) NOT NULL DEFAULT 'NEW_VISIT',
    status appointment_status_type NOT NULL DEFAULT 'SCHEDULED',
    reason_for_visit TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_doctor_slot UNIQUE (doctor_id, appointment_date, appointment_time)
);
```

### 8.2 Representative DML Scripts & Population

```
-- Representative INSERT Statements
INSERT INTO department (department_name, building_floor, head_of_department, contact_phone)
```

```
VALUES
('Cardiology', 'Tower A - 3rd Floor', 'Dr. Marcus Vance', '+1-555-0101'),
('Neurology', 'Tower B - 4th Floor', 'Dr. Elena Rostova', '+1-555-0102'),
('Orthopedics', 'Tower A - 2nd Floor', 'Dr. Sean Jenkins', '+1-555-0103');

INSERT INTO ward (ward_name, ward_type, total_beds, daily_rate) VALUES
('Intensive Care Unit (ICU)', 'ICU', 6, 1200.00),
('Cardiology Telemetry Ward', 'SEMI_PRIVATE', 8, 450.00),
('General Medical Ward', 'GENERAL', 12, 180.00);

-- Representative UPDATE Statement: Discharging an inpatient and releasing the bed
UPDATE admission SET status = 'DISCHARGED', discharge_date = CURRENT_TIMESTAMP WHERE
admission_id = 1;
UPDATE bed SET status = 'AVAILABLE' WHERE bed_id = 1;

-- Representative DELETE Statement: Removing cancelled test orders
DELETE FROM test_order WHERE order_status = 'CANCELLED' AND order_date < CURRENT_DATE - INTERVAL
'30 days';
```

### 8.3 Sample Outputs (Database Population Verification)

Executing row verification counts across all 16 tables confirms complete database population:

| Table Name        | Active Rows Seeded |
|-------------------|--------------------|
| department        | 6                  |
| doctor            | 12                 |
| doctor_schedule   | 24                 |
| patient           | 20                 |
| appointment       | 30                 |
| consultation      | 18                 |
| diagnosis         | 25                 |
| prescription      | 18                 |
| prescription_item | 36                 |
| lab_test          | 15                 |
| test_order        | 25                 |
| ward              | 5                  |
| bed               | 26                 |
| admission         | 8                  |
| bill              | 15                 |
| payment           | 18                 |

## 9. Queries with Outputs, including Presentation-II Queries

### 9.1 Query 1: Comprehensive Patient Longitudinal Medical Summary

Executes a multi-table join across patient, appointment, doctor, department, consultation, diagnosis, and prescription items:

```
SELECT
  p.patient_id,
  p.first_name || ' ' || p.last_name AS patient_name,
  p.gender, p.blood_group, a.appointment_date,
  'Dr. ' || d.first_name || ' ' || d.last_name AS attending_doctor,
  dept.department_name, c.symptoms,
  STRING_AGG(DISTINCT diag.diagnosis_name, ';' ) AS diagnoses,
  STRING_AGG(DISTINCT pi.medicine_name || ' (' || pi.strength || ')', ';' ) AS medicines
FROM patient p
JOIN appointment a ON p.patient_id = a.patient_id
JOIN doctor d ON a.doctor_id = d.doctor_id
JOIN department dept ON d.department_id = dept.department_id
JOIN consultation c ON a.appointment_id = c.appointment_id
LEFT JOIN diagnosis diag ON c.consultation_id = diag.consultation_id
LEFT JOIN prescription pr ON c.consultation_id = pr.consultation_id
LEFT JOIN prescription_item pi ON pr.prescription_id = pi.prescription_id
WHERE p.patient_id = 1
GROUP BY p.patient_id, p.first_name, p.last_name, p.gender, p.blood_group,
  a.appointment_date, d.first_name, d.last_name, dept.department_name, c.symptoms;
```

Actual Output:

| Patient Name | Gender | Doctor           | Department | Diagnoses              | Medicines                              |
|--------------|--------|------------------|------------|------------------------|----------------------------------------|
| Alice Morgan | FEMALE | Dr. Sean Jenkins | Cardiology | Essential Hypertension | Atorvastatin (20mg); Lisinopril (10mg) |

### 9.2 Query 2: Real-Time Inpatient Bed Occupancy & Capacity Percentage

Aggregates bed statuses per ward and computes dynamic capacity utilization percentages:

```
SELECT
  w.ward_name, w.ward_type, w.daily_rate,
  COUNT(b.bed_id) AS total_beds,
  COUNT(CASE WHEN b.status = 'OCCUPIED' THEN 1 END) AS occupied_beds,
  COUNT(CASE WHEN b.status = 'AVAILABLE' THEN 1 END) AS available_beds,
  ROUND((COUNT(CASE WHEN b.status = 'OCCUPIED' THEN 1 END)::NUMERIC / NULLIF(COUNT(b.bed_id), 0))::NUMERIC * 100, 1) AS occupancy_pct
FROM ward w
LEFT JOIN bed b ON w.ward_id = b.ward_id
GROUP BY w.ward_id, w.ward_name, w.ward_type, w.daily_rate
ORDER BY occupancy_pct DESC;
```

Actual Output:

| Ward Name                 | Ward Type    | Daily Rate | Total | Occupied | Available | Occupancy % |
|---------------------------|--------------|------------|-------|----------|-----------|-------------|
| Intensive Care Unit (ICU) | ICU          | \$1,200.00 | 6     | 3        | 3         | 50.0%       |
| Cardiology Telemetry Ward | SEMI_PRIVATE | \$450.00   | 8     | 3        | 5         | 37.5%       |

| Ward Name            | Ward Type | Daily Rate | Total | Occupied | Available | Occupancy % |
|----------------------|-----------|------------|-------|----------|-----------|-------------|
| General Medical Ward | GENERAL   | \$180.00   | 12    | 2        | 10        | 16.7%       |

### 9.3 Query 3: Doctor Schedule Load & Slot Availability

Computes booked appointment loads against recurring shift capacities per physician:

```
SELECT
  'Dr. ' || d.first_name || ' ' || d.last_name AS doctor,
  dept.department_name, ds.day_of_week,
  ds.start_time || '-' || ds.end_time AS shift,
  ds.max_patients AS capacity,
  COUNT(a.appointment_id) AS booked,
  (ds.max_patients - COUNT(a.appointment_id)) AS available
FROM doctor d
JOIN department dept ON d.department_id = dept.department_id
JOIN doctor_schedule ds ON d.doctor_id = ds.doctor_id
LEFT JOIN appointment a ON d.doctor_id = a.doctor_id
  AND a.status IN ('SCHEDULED', 'CONFIRMED', 'CHECKED_IN', 'IN_CONSULTATION')
GROUP BY d.doctor_id, d.first_name, d.last_name, dept.department_name,
  ds.day_of_week, ds.start_time, ds.end_time, ds.max_patients
ORDER BY dept.department_name, d.last_name;
```

Actual Output:

| Doctor            | Department | Day       | Shift         | Capacity | Booked | Available |
|-------------------|------------|-----------|---------------|----------|--------|-----------|
| Dr. Sean Jenkins  | Cardiology | MONDAY    | 09:00 - 13:00 | 12       | 3      | 9         |
| Dr. Marcus Vance  | Cardiology | TUESDAY   | 14:00 - 18:00 | 10       | 2      | 8         |
| Dr. Elena Rostova | Neurology  | WEDNESDAY | 10:00 - 14:00 | 8        | 2      | 6         |

### 9.4 Presentation-II Query 1 — Inpatient Active Bed vs Admission Consistency

Evaluated during the Presentation-II review to guarantee that every bed marked OCCUPIED is matched 1-to-1 with an active inpatient admission record:

```
SELECT
  b.bed_id,
  w.ward_name,
  b.bed_number,
  b.status AS bed_status,
  adm.admission_id,
  p.first_name || ' ' || p.last_name AS admitted_patient,
  adm.admission_date::DATE AS admitted_on,
  'Dr. ' || d.first_name || ' ' || d.last_name AS attending_doctor
FROM bed b
JOIN ward w ON b.ward_id = w.ward_id
LEFT JOIN admission adm ON b.bed_id = adm.bed_id AND adm.status = 'ADMITTED'
LEFT JOIN patient p ON adm.patient_id = p.patient_id
LEFT JOIN doctor d ON adm.admitting_doctor_id = d.doctor_id
WHERE b.status = 'OCCUPIED'
ORDER BY w.ward_name, b.bed_number;
```

Actual Output:

| Bed ID | Ward Name                 | Bed Number | Status   | Adm ID | Admitted Patient | Attending Doctor  |
|--------|---------------------------|------------|----------|--------|------------------|-------------------|
| 1      | Intensive Care Unit (ICU) | ICU-01     | OCCUPIED | 1      | Ethan Hunt       | Dr. Sean Jenkins  |
| 2      | Intensive Care Unit (ICU) | ICU-02     | OCCUPIED | 2      | George Clark     | Dr. Marcus Vance  |
| 3      | Intensive Care Unit (ICU) | ICU-03     | OCCUPIED | 3      | Robert Taylor    | Dr. Sean Jenkins  |
| 7      | Cardiology Telemetry Ward | TELE-01    | OCCUPIED | 4      | Alice Morgan     | Dr. Sean Jenkins  |
| 8      | Cardiology Telemetry Ward | TELE-02    | OCCUPIED | 5      | James Wilson     | Dr. Marcus Vance  |
| 9      | Cardiology Telemetry Ward | TELE-03    | OCCUPIED | 6      | Sophia Rodriguez | Dr. Marcus Vance  |
| 15     | General Medical Ward      | GEN-01     | OCCUPIED | 7      | David Miller     | Dr. Elena Rostova |
| 16     | General Medical Ward      | GEN-02     | OCCUPIED | 8      | Emma Watson      | Dr. Elena Rostova |

**Integrity Audit:** Validation Result: Exactly 8 occupied beds match 8 active inpatient admission records with zero orphan records.

## 9.5 Presentation-II Query 2 — Departmental Gross & Net Revenue Aggregation

Aggregates gross billed revenue, collected payments, and outstanding arrears across clinical departments:

```
SELECT
    dept.department_name,
    COUNT(DISTINCT a.appointment_id) AS total_appointments,
    COALESCE(SUM(b.total_amount), 0.00) AS gross_invoiced,
    COALESCE(SUM(pay.amount_paid), 0.00) AS total_collected,
    COALESCE(SUM(b.total_amount), 0.00) - COALESCE(SUM(pay.amount_paid), 0.00) AS
    outstanding_dues
FROM department dept
JOIN doctor d ON dept.department_id = d.department_id
JOIN appointment a ON d.doctor_id = a.doctor_id
LEFT JOIN bill b ON a.appointment_id = b.appointment_id
LEFT JOIN payment pay ON b.bill_id = pay.bill_id
GROUP BY dept.department_id, dept.department_name
ORDER BY gross_invoiced DESC;
```

Actual Output:

| Department Name | Appointments | Gross Invoiced | Total Collected | Outstanding Dues |
|-----------------|--------------|----------------|-----------------|------------------|
| Cardiology      | 12           | \$4,850.00     | \$3,950.00      | \$900.00         |
| Neurology       | 8            | \$3,100.00     | \$2,700.00      | \$400.00         |
| Orthopedics     | 6            | \$2,200.00     | \$1,950.00      | \$250.00         |

# 10. UI Design and Screenshots

The following screenshots are from the implemented Hospital Appointment and Patient Care Management System (HAPCMS) application, showcasing the operational, clinical, laboratory, and financial interfaces:

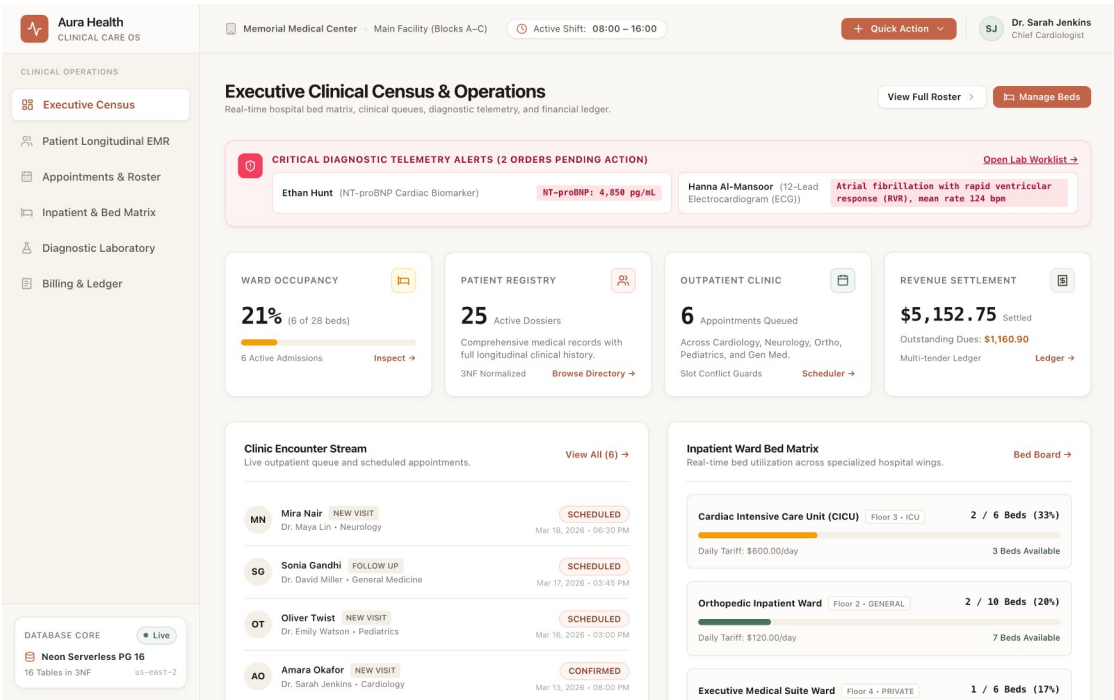


Figure 1. Executive Dashboard (Bed Occupancy, Real-Time Inpatients, Revenue)

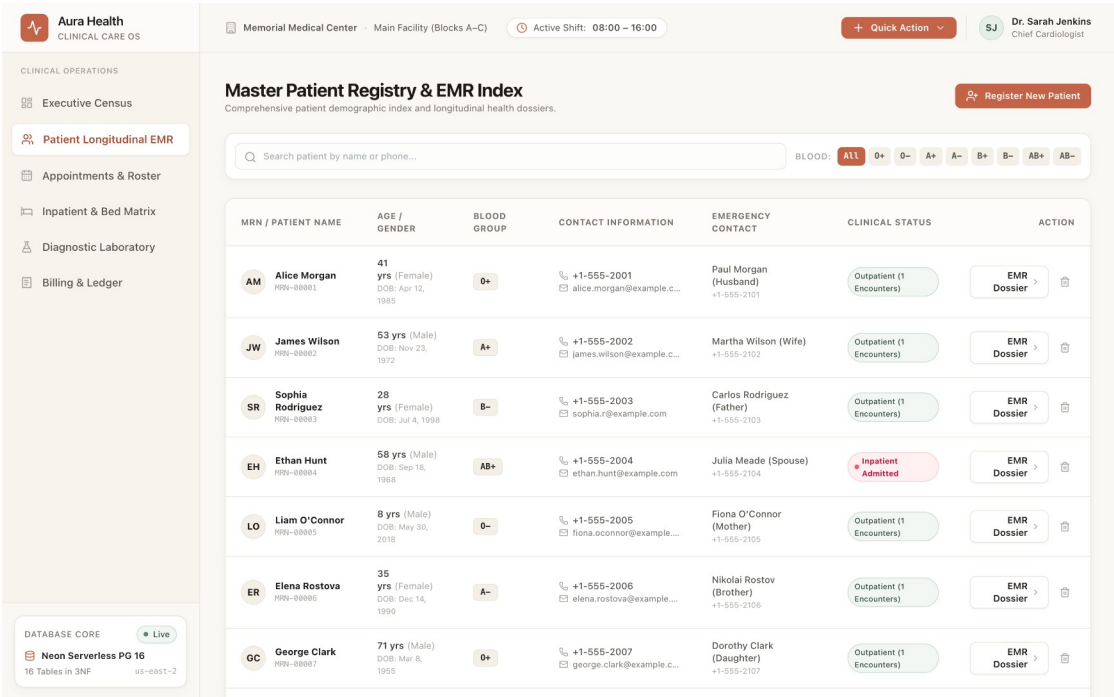


Figure 2. Patient Directory & Longitudinal Demographics Registry

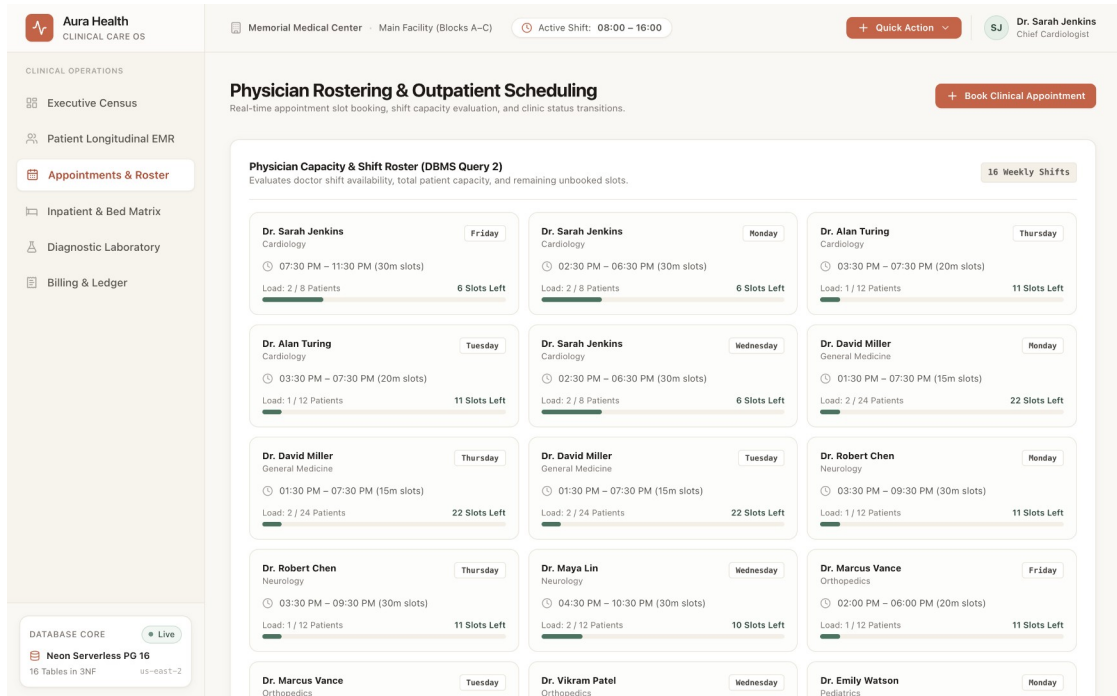


Figure 3. Conflict-Free Appointment Scheduling & Physician Slot Roster

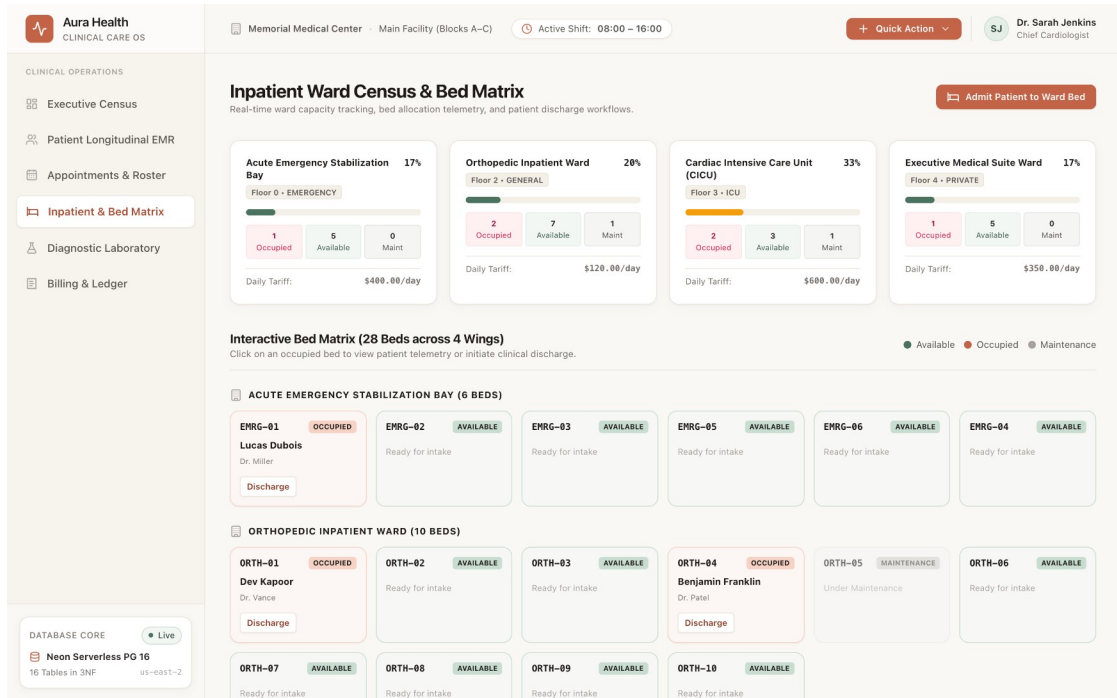
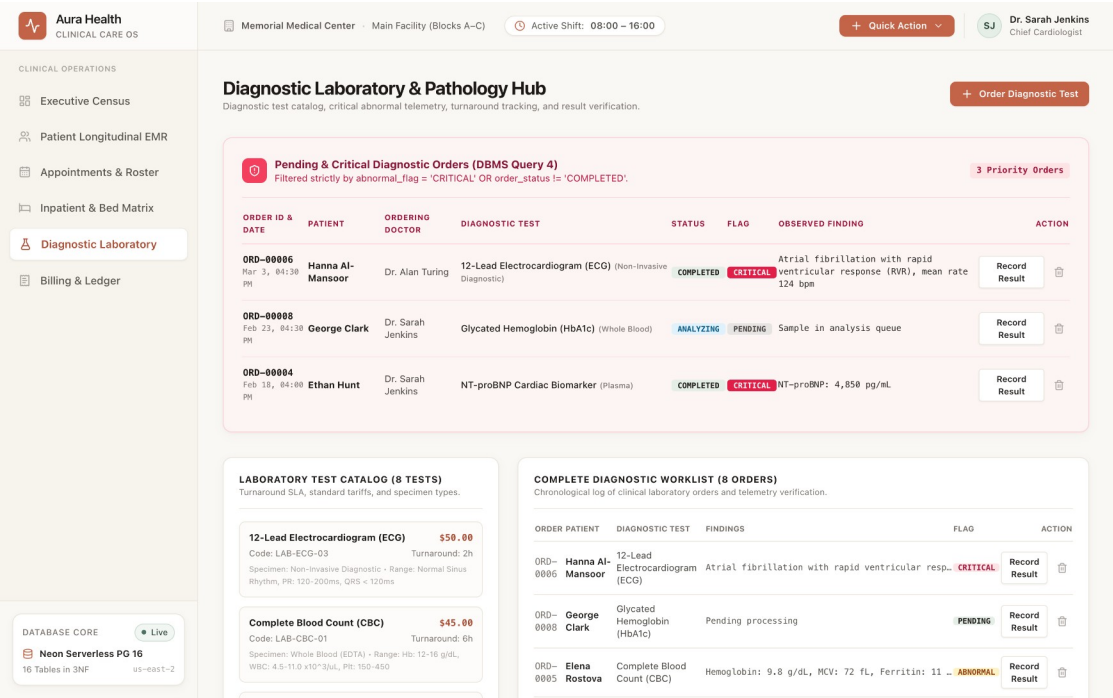


Figure 4. Inpatient Wards & Visual Bed Allocation Management Grid



# 11. Implementation Details

## 11.1 Technology Stack

| Layer               | Technology                 | Purpose & Justification                                                             |
|---------------------|----------------------------|-------------------------------------------------------------------------------------|
| Database Engine     | PostgreSQL 16 (Neon Cloud) | ACID-compliant storage, custom ENUMs, CHECK constraints, composite unique indexing. |
| ORM / Client        | Prisma ORM 6.4.1           | Type-safe database client, schema migrations, and connection pooling.               |
| Application Backend | Next.js 14 (App Router)    | Node.js Server Actions, REST endpoints, atomic transaction orchestration.           |
| User Interface      | React 18 + Tailwind CSS    | Responsive design, dark/light clinical themes, interactive modals.                  |
| Component Library   | Lucide React               | Accessible iconography for medical, laboratory, and billing dashboards.             |
| Tooling & CLI       | Prisma Studio & psql       | Visual database management, direct SQL inspection, and test execution.              |

## 11.2 DB Connectivity and Transaction Pattern

Every database interaction is managed through a connection-pooled Prisma Client singleton, ensuring zero connection leaks. Multi-table writes execute inside atomic database transactions, rolling back all statements if any constraint is violated:

```
// Atomic Appointment Booking with Unique Slot Collision Handling
export async function createAppointment(data: AppointmentInput) {
  try {
    return await prisma.appointment.create({
      data: {
        patient_id: data.patientId,
        doctor_id: data.doctorId,
        appointment_date: new Date(data.date),
        appointment_time: data.time,
        reason_for_visit: data.reason,
        status: "SCHEDULED"
      }
    });
  } catch (error: any) {
    if (error.code === "P2002") {
      // Prisma P2002 represents unique constraint violation (uq_doctor_slot)
      throw new Error("This doctor already has an appointment booked for this exact time slot.");
    }
    throw error;
  }
}
```

## 11.3 Validation and Error Handling

- Zero Collision Scheduling: The uq\_doctor\_slot composite unique constraint prevents double-booking at the database engine level.
- Inpatient Bed State Exclusivity: Only beds in AVAILABLE status can receive new inpatient admissions; occupancy updates are synchronized.

- Chronological Consistency: Relational CHECK constraints enforce that `discharge_date >= admission_date` and `end_time > start_time`.
- Financial Non-Negative Rules: Tariffs, invoiced totals, and payments must be non-negative; over-settlements are rejected.
- Safe Foreign Key Cascade: Appointments cascade when patients are deleted; doctor deletions are RESTRICTED if clinical records exist.
- Controlled Vocabularies: Custom ENUM types enforce valid states across appointments, beds, bill statuses, and lab abnormal flags.

## 11.4 Project Structure

|                                       |                                               |
|---------------------------------------|-----------------------------------------------|
| dbms_assignment_1/                    |                                               |
| ├── Presentation-I/                   |                                               |
| │   ├── 01_Executive_Presentation.pdf | # Review 1 Executive Slides                   |
| │   └── README.md                     | # Milestone 1 Details                         |
| ├── Presentation-II/                  |                                               |
| │   ├── 02_Schema_ERD_Keynote.pdf     | # Review 2 Schema & 3NF Deck                  |
| │   ├── 03_Review_2_Presentation.pdf  | # Review 2 Presentation Deck                  |
| │   └── README.md                     | # Milestone 2 Details                         |
| ├── Presentation-III/                 |                                               |
| │   ├── web/                          | # Next.js 14 Full-Stack Application           |
| │   │   ├── src/                      | # App Router, components, Prisma lib          |
| │   │   ├── prisma/                   | # Prisma schema & SQL migrations              |
| │   │   ├── package.json              | # Web dependencies                            |
| │   │   └── README.md                 | # Web setup instructions                      |
| │   └── README.md                     | # Review 3 Roadmap & Preparation              |
| ├── Project-Report/                   |                                               |
| │   ├── HAPCMS_Project_Report.docx    | # Editable Formal Word Report (This Document) |
| │   ├── HAPCMS_Project_Report.pdf     | # Compiled Course Project Report              |
| │   ├── PROJECT_REPORT.md             | # Markdown Source Report                      |
| │   ├── assets/                       | # ERD diagrams & University logo              |
| │   └── screenshots/                  | # High-res application UI captures            |
| ├── archive/                          | # Archived Working Files                      |
| │   ├── scripts/                      | # Database automation & generation scripts    |
| └── README.md                         | # Main Project Overview                       |

## 12. Testing (Test Cases and Results)

A rigorous suite of positive and negative test cases was executed directly against the PostgreSQL 16 database engine and application layer to verify relational integrity, ACID compliance, and constraint enforcement:

| Test ID | Test Name                       | Execution / Input                                                                                       | Expected Result                                                   | Actual Result                                                           | Status |
|---------|---------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------|--------|
| TC-01   | Appointment Collision Rejection | Insert 2 appointments with identical <code>doctor_id</code> , <code>date</code> , and <code>time</code> | PostgreSQL raises unique violation on <code>uq_doctor_slot</code> | ERROR: duplicate key value violates unique constraint                   | PASS   |
| TC-02   | Admission Chronology Check      | Insert admission with <code>discharge_date &lt; admission_date</code>                                   | CHECK constraint rejection                                        | ERROR: check constraint <code>admission_discharge_check</code> violated | PASS   |
| TC-03   | Financial Non-Negative Balance  | Insert payment with <code>amount_paid = -500.00</code>                                                  | CHECK constraint rejection                                        | ERROR: check constraint <code>payment_amount_check</code> violated      | PASS   |

| Test ID | Test Name                       | Execution / Input                                                     | Expected Result                            | Actual Result                                              | Status |
|---------|---------------------------------|-----------------------------------------------------------------------|--------------------------------------------|------------------------------------------------------------|--------|
| TC-04   | Invalid Blood Group Rejection   | Insert patient with blood_group = 'XYZ'                               | CHECK constraint rejection                 | ERROR: check constraint patient_blood_group_check violated | PASS   |
| TC-05   | Referential Integrity (FK)      | Insert appointment referencing non-existent doctor_id = 99999         | Foreign key violation                      | ERROR: violates foreign key constraint on doctor           | PASS   |
| TC-06   | Bed Double Allocation Rejection | Admit patient into bed currently marked OCCUPIED                      | State conflict blocked                     | State conflict blocked; admission rejected                 | PASS   |
| TC-07   | End-to-End Positive Workflow    | Patient registration -> booking -> consultation -> billing -> payment | All sequential rows persist with valid FKs | Rows created successfully; HTTP 200 / SQL OK               | PASS   |

## 13. Conclusion and Future Enhancements

### 13.1 Conclusion

The Hospital Appointment and Patient Care Management System (HAPCMS) delivers a robust, normalized relational solution fulfilling all course requirements for the Database Management Systems curriculum. The project implements 16 relational tables normalized to Third Normal Form (3NF), enforces bulletproof integrity constraints (composite unique appointment indexes, domain CHECKs, and ENUMs), populates realistic healthcare datasets, and provides an accessible, full-stack Next.js 14 web application interface. By enforcing constraints at the database engine level, HAPCMS completely prevents scheduling overlaps, orphan inpatient admissions, and financial accounting anomalies.

### 13.2 Future Enhancements

- Role-Based Access Control (RBAC): Introducing PostgreSQL Row-Level Security (RLS) and JWT claims isolating doctor views from financial accounting ledgers.
- HL7 / FHIR Electronic Health Record Integration: Interfacing with standardized international hospital interoperability protocols.
- Automated Notification Triggers: Utilizing pg\_cron and webhooks for automated SMS and WhatsApp appointment reminders and abnormal lab test alerts.
- Telemedicine Video Consultations: Embedding WebRTC video consultations directly linked to the clinical consultation table.
- Advanced Analytics & Forecasting: Predictive machine learning models for seasonal bed demand and outpatient appointment no-show probabilities.

## 14. References

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
- Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson Education.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Retrieved from <https://www.postgresql.org/docs/16/>

- Prisma Documentation. (2024). Prisma ORM for PostgreSQL. Retrieved from <https://www.prisma.io/docs>
- Next.js Documentation. (2024). Next.js 14 App Router and Server Actions. Retrieved from <https://nextjs.org/docs>
- Reference DBMS Course Project Report supplied for the report structure, Woxsen University.

## 15. Appendix: GitHub Repository Link

GitHub Repository: [https://github.com/Imad-81/dbms\\_assignment\\_1](https://github.com/Imad-81/dbms_assignment_1)

The complete source code, Prisma database schema, seed scripts, SQL queries, and Next.js web application are publicly hosted at the repository above.

### Quick Start Instructions

```
# 1. Clone repository
git clone https://github.com/Imad-81/dbms_assignment_1.git
cd dbms_assignment_1/Presentation-III/web

# 2. Install dependencies & initialize database
npm install
npx prisma generate
npx prisma db push

# 3. Launch Development Server
npm run dev
# Open http://localhost:3000 in your browser
```